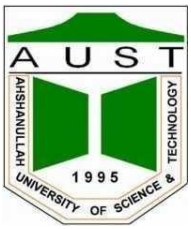
Ahsanullah University of Science & Technology
Department of Computer Science & Engineering



3D Interactive Reading Room

Computer Graphics Lab (CSE 4204)

Final Project Report

Submitted By:	
Mirza Naeem Beg	20210204033
Salim Ullah Sadik	20200204085

1. Problem Analysis

1.1 Project Requirements

The main goal of this project is to create an interactive 3D reading room using Three.js and WebGL. In this scene:

- Custom shaders are used to give the floor a unique visual effect.
- Different types of lighting are added, including a moving light that creates realistic shadows.
- A perspective camera is used, allowing the user to move around the room like a first-person view.
- Textures (images) are applied to walls, floor, and furniture to make them look realistic.
- The room includes 3D objects like a large table, six chairs, and four bookshelves filled with books.
- The user can move around using the keyboard.
- By clicking with the mouse, the table's wood texture can be changed during runtime.

1.2 Core Integration

All objects in the scene are created using basic Three.js shapes. The room is built as a box that is rendered from the inside, allowing the interior walls to be visible. The floor is a separate flat surface that uses a custom shader to create a unique visual effect. Four bookshelves are constructed using simple box and cylinder shapes and are filled with books of different sizes and colors to make them look more natural. The central table is made from a flat top, four slightly tapered legs, side support boards, and connecting braces. Six chairs are placed around the table, each designed with a seat, back slats, a top rail, armrests with supports, and legs connected by braces. The camera is set to a first-person view with a 72° field of view, positioned at eye level (1.7 meters), and restricted within the room boundaries. Lighting includes a soft ambient light for overall brightness, a directional light for general illumination, a cool-colored point light for contrast, and a moving warm point light that creates realistic soft shadows. Textures are applied to most objects using standard materials for realistic rendering, while the floor uses a custom shader instead.

1.3 Technical Challenges

The main challenge was displaying the floor texture correctly using a custom ShaderMaterial, since it ignores built-in settings like texture.repeat. This was solved by manually tiling the texture in the fragment shader using repeatedUV = fract(vUv * uTexRepeat), with uTexRepeat passed as a uniform.Another issue was Z-fighting between the floor and the room base, as both were positioned at y = −3, causing flickering. This was fixed by slightly raising the floor to y = −2.999 and enabling polygonOffset (factor = −1, units = −1).A third problem involved variable name conflicts, where variables like legY, lx, and lz were declared both globally and inside a function. This was resolved by renaming the global variables to tLegY, tLX, and tLZ.

2. User Interaction

2.1 Key Interaction

Keyboard input is handled using keydown and keyup events, which store pressed keys in a dictionary. In each animation frame, handleCamera() reads this input to move the camera: W/ArrowUp moves forward, S/ArrowDown backward, A/ArrowLeft left, and D/ArrowRight right, while Q and E rotate the camera horizontally.The forward direction is taken from camera.getWorldDirection() with the Y component removed and normalized, and the right direction is computed using a cross product. The camera position is then clamped within ±7 meters on the X and Z axes to keep it inside the room, and the height is fixed at 1.7 meters for an eye-level view.

2.2 Mouse Interaction

A click event listener converts the mouse position into Normalised Device Coordinates (NDC) and uses raycaster.setFromCamera(mouse, camera) to detect intersections with the table meshes. If the clicked object is the tabletop, its texture is switched between two wood options (wood.jpg and wood2.jpg) by toggling a boolean, and material.needsUpdate is set to refresh it on the GPU. A short emissive flash highlights the change, giving instant visual feedback. This allows the user to compare different table finishes in real time without reloading the scene.

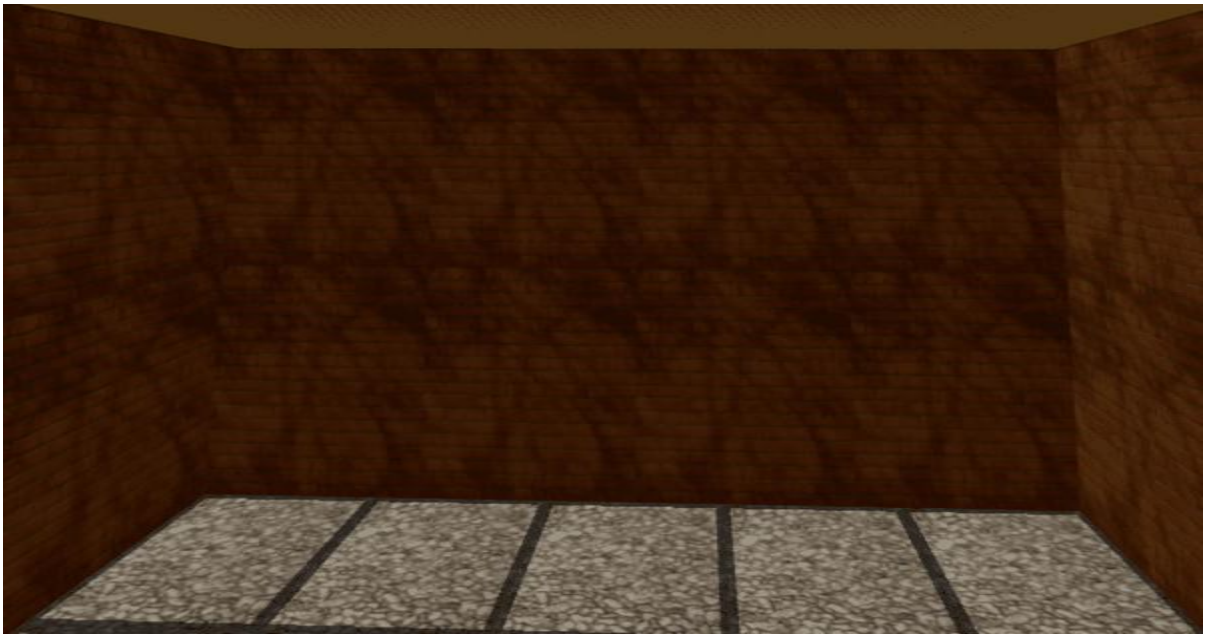
3. Design and Aesthetic Aspects

3.1 Visual Elements

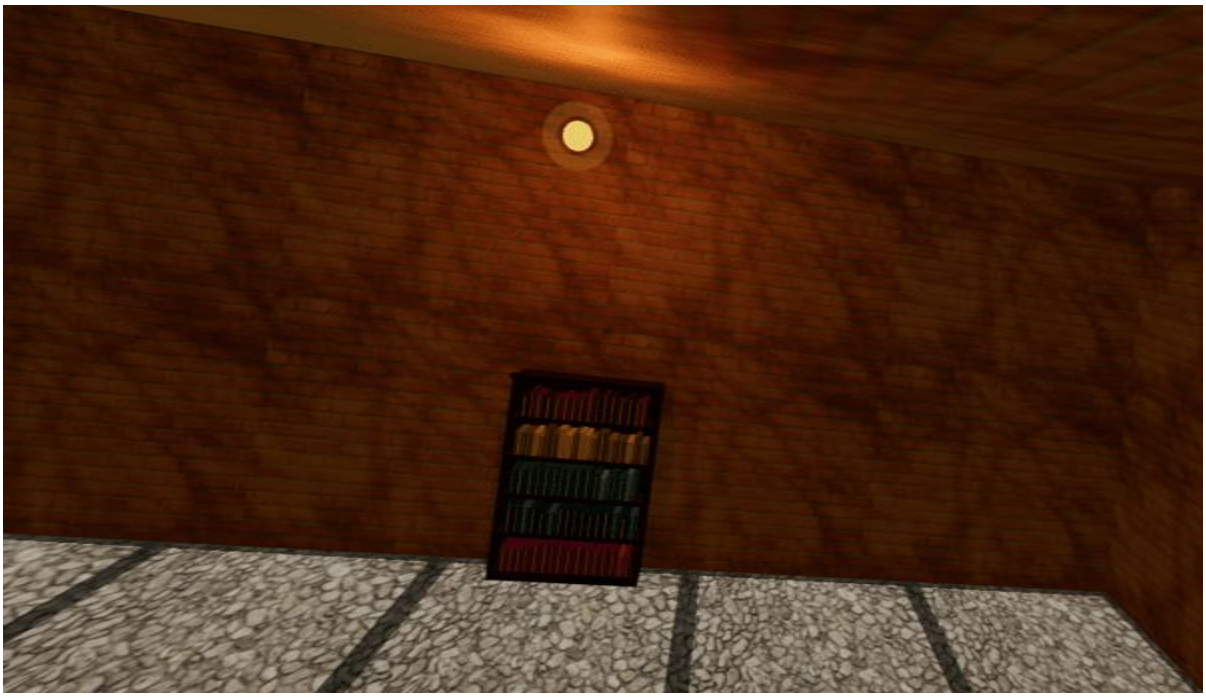
The walls use a brick texture (wall.jpg) tiled at 4×2 and tinted with a warm sandy color using MeshStandardMaterial, allowing realistic lighting response. The floor uses a custom ShaderMaterial, where floor.jpg is tiled 5×5 manually in GLSL, with a procedural grout grid added by darkening the tile edges for extra detail.Wood textures (wood.jpg and wood2.jpg) are applied to the table and chairs using MeshStandardMaterial with moderate roughness and low metalness, giving a natural wood look. The lighting setup includes a warm ambient light for overall visibility, a soft directional light from above, a cool blue point light for contrast, and a main warm point light that moves around, changes intensity, and casts soft shadows. Finally, ACESFilmic tone mapping with slight exposure is used to give the scene a more cinematic appearance.

3.2 Snapshots

Snapshot 1 – Default front view showing the conference table, eight chairs, and four bookshelves.



Snapshot 2 – Close-up of the floor with the tile texture and grout grid clearly distinguishable from the brick wall texture above it.



Snapshot 3 – Orbiting light passing behind the bookshelves, casting dynamic shadows across the furniture.

Snapshot 4 – Table surface after a mouse click showing the alternate wood2 texture with a brief emissive flash.

4. Code Implementation

4.1 Software Platform

HTML5, CSS3, JavaScript (ES6 Modules), Three.js v0.160.0, WebGL (via Three.js WebGLRenderer), GLSL ES 1.0 (custom vertex and fragment shaders).

4.2 Custom Shaders

The vertex shader uses built-in attributes like position, uv, and transformation matrices. It passes the UV coordinates (vUv) and world position (vWorldPos) to the fragment shader, and calculates the final vertex position using the standard projection and model-view transformation.The fragment shader receives uniforms such as the light position, time, floor texture, and texture repeat. It tiles the texture by modifying the UVs, adds a grout grid effect using step(), and darkens those grout lines for detail. Lighting is applied with smooth attenuation and a slight specular highlight, and the final color is then output to the screen.

4.3 Project Feature Table:

Attach a table with all your required/additional features and classify them into three categories: Implemented, Partially Implemented and Not Implemented.

#	Features	Status
1	Custom GLSL Vertex & Fragment Shader (floor texture + grout grid + lighting)	Implemented
2	Multiple Light Sources (Ambient, Directional, cool PointLight, animated orbiting PointLight with shadow)	Implemented
3	Perspective Camera with First-Person Navigation (WASD + Arrow keys, Q/E rotate, clamped bounds)	Implemented
4	Texture Mapping (wall brick, floor tile with GLSL repeat, wood table + chairs, swappable table texture)	Implemented
5	Mouse Raycaster Interaction (click tabletop to swap wood texture with emissive flash feedback)	Implemented

Table 01: Project Feature Table

5. Contribution

Mirza Naeem Beg (ID: 20210204033) was responsible for designing the 3D scene structure and building all geometry, including the room, ceiling, bookshelves with procedurally generated books, the conference table, and detailed chairs. He also implemented the keyboard-based first-person camera movement, boundary constraints, and the animated orbiting light with shadows.

Salim Ullah Sadik (ID: 20200204085) handled the custom GLSL shader for the floor, including texture tiling, grout effects, lighting calculations, and specular highlights. He also developed the mouse interaction for texture switching, fixed Z-fighting issues, managed texture loading, applied tone mapping, and added accent lighting.

Both members contributed equally to debugging, testing, and final integration of the complete scene.

6. Conclusion

This project successfully developed an interactive 3D reading room using Three.js and WebGL, combining features such as custom shaders, multiple lighting systems, texture mapping, and real-time user interaction. The use of first-person camera movement and raycaster-based interaction created an immersive and engaging experience. Key technical challenges, including shader texture handling and Z-fighting, were effectively resolved, improving both stability and visual quality. Overall, the project demonstrates a solid understanding of modern computer graphics and provides a strong foundation for future work in real-time rendering and advanced 3D applications.